

Support HTML modulaire

Master 2

6 jours

3 TP progressifs

Java / Spring Boot

TP3 — Résilience + observabilité + sécurité

Objectifs pédagogiques

Finaliser l'architecture distribuée avec des préoccupations de production réalistes.

Temps estimé

≈ 6h incluant démos et quiz

Prérequis

TP2 terminé + Jour 5

Table des matières

- **Objectifs**
- Énoncé
- Étapes
- Code
- Pièges
- Évaluation
- Corrigé

Objectifs pédagogiques

- Ajouter résilience et observabilité.
- Sécuriser les APIs.
- Documenter l'architecture finale.
- Introduire un échange asynchrone simple.

Énoncé

Sur la base du TP2 :

- ajouter Resilience4j sur l'appel catalogue ;
- ajouter logs corrélés et métriques Actuator ;
- sécuriser les endpoints avec JWT ;
- publier un événement `loan-created` pour notification ;
- produire un mini dossier d'architecture final.

Étapes détaillées

1. Configurer timeout, retry, circuit breaker.

2. Ajouter un filtre de corrélation d'ID.
3. Exposer `/actuator/health` et `/actuator/prometheus`.
4. Configurer sécurité resource server JWT.
5. Documenter le flux synchrone et le flux événementiel.

Extraits de code commentés

```
resilience4j:
  circuitbreaker:
    instances:
      catalogService:
        slidingWindowSize: 10
        failureRateThreshold: 50
  retry:
    instances:
      catalogService:
        maxAttempts: 3
```

```
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String correlationId = Optional.ofNullable(request.getHeader("X-Correlation-Id"))
            .orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", correlationId);
        response.addHeader("X-Correlation-Id", correlationId);
        try {
            filterChain.doFilter(request, response);
        } finally {
            MDC.clear();
        }
    }
}
```

Pièges fréquents

- Ajouter retry + timeout sans stratégie de fallback.
- Sécuriser l'entrée mais pas les échanges internes sensibles.
- Ne produire aucun schéma ni ADR alors que l'architecture a changé.
- Faire un événement sans consommateur clairement défini.

Critères d'évaluation

Axe	Attendu
Résilience	Timeout/retry/circuit breaker cohérents
Observabilité	Logs corrélés + métriques + health
Sécurité	JWT, rôles, endpoints protégés
Documentation	Dossier d'architecture et ADR
Maturité	Capacité à expliquer les compromis

Corrigé détaillé

Le corrigé attendu doit rendre visible la différence entre “ça marche” et “c’est exploitable en conditions réelles”.

Ce que l’étudiant doit avoir compris

Une architecture distribuée n’est aboutie que si elle est observable, sécurisée et documentée, pas seulement découpée.